CYBERSIREN — SYSTEM ARCHITECTURE SPECIFICATION

Document: ARCH-SPEC-v2.1 **Date:** 2026-03-11 **Status:** Implementation Baseline — Approved Architecture with Tracked Deltas
Classification: Internal / Graduation Project

◘ Service (thick border = entry point)	☐ Kafka Topic	☐ Database Operation	= **R** Read	= **W** Write	**R/W** = Both	All scores: integer 0–100 confidence: float 0.0–1.0	All

1. COMPLETE PIPELINE — STEP-BY-STEP DATA FLOW

Canonical Identifier Strategy:
email_id (UUIDv7) = logical identifier. Assigned by SVC-01, carried in all Kafka messages and Redis keys, used as partition key.
internal_id (BIGSERIAL) = physical DB primary key. Auto-generated on INSERT. Part of composite PK (internal_id, fetched_at) required by partitioning.
fetched_at = partition key for the emails table. Part of composite PK. Not part of logical identity.
message_id (RFC 5322) = deduplication key scoped per org (org_id, message_id). Not globally unique.
Mapping: emails.scored message includes both email_id and (internal_id, fetched_at) so SVC-08 can write to DB without an extra lookup.

Data Role Taxonomy: PostgreSQL = source of record (authoritative state). **Redis** = ephemeral coordination/cache (lossy, rebuildable from DB). **Kafka** = transport/replay (bounded retention 24h–7d). **Materialized Views** = read optimization (never the source of truth). **Object Storage (S3/MinIO)** = binary retention (attachment files, 90-day TTL).

Each numbered step shows: the service performing work, the Kafka topic it produces to (if any), and the database operations (if any). Every field in every message is listed.

1

EMAIL INGESTION SERVICE (Go) — *SVC-01*
Receives raw email from one of 5 adapters (see §2.1). Normalizes to internal ParsedEmail envelope. Assigns email_id (UUIDv7), org_id (from API key auth), fetched_at (now). Performs idempotent dedup: checks (org_id, message_id) in Redis set (TTL 7d) → if present, discard. On cache miss, falls back to DB unique constraint uq_emails_org_message_id.

→ **emails.raw**	**R** Redis: dedup:{org_id}:{message_id}
Key: email_id · 6 partitions · 48h retention	No PostgreSQL writes at this stage.

Message schema — emails.raw:
{ email_id, org_id, fetched_at, raw_rfc822: <full email bytes base64>, source_adapter: "gmail"|"outlook"|"imap"|"api"|"custom", api_key_id }

2

EMAIL PARSER SERVICE (Go) — *SVC-02*
Consumes emails.raw. Parses MIME structure. Produces 5 output topics (fan-out). Extracts: all URLs from HTML (href/src/action/meta-refresh) + plain text (regex). Hashes each attachment (SHA256 + MD5 + SHA1), computes Shannon entropy, detects actual file type via magic bytes. Extracts all header authentication results. Strips HTML → clean plain text for NLP. Builds Analysis Plan listing which downstream scores to expect based on email contents. **Writes initial email record + junction tables to PostgreSQL.**

→ **5 topics:**	**W** emails (INSERT: 26 metadata cols from 001+002)
analysis.urls · analysis.headers · analysis.attachments · analysis.text · analysis.plans	**U** enriched_threats (UPSERT bare: url, domain, tld — for email_urls FK)
	W email_urls (INSERT: email_id, threat_id→enriched_threats.id, visible_text)
	U attachment_library (UPSERT by sha256: sha256, md5, sha1, size_bytes, entropy)
	W email_attachments (INSERT: email_id, attachment_id, filename, content_type)
	W email_recipients (INSERT: email_id, org_id, address, recipient_type)

Output message schemas from Email Parser:

analysis.urls (6 partitions · 24h · Key: email_id):
{ email_id, org_id, urls: [{ url: string, visible_text: string, position: "body"|"header", html_context: "href"|"src"|"action"|"plain_text" }] }

analysis.headers (6 partitions · 24h · Key: email_id):
{ email_id, org_id, sender_email, sender_domain, sender_name, reply_to_email, return_path, originating_ip, x_originating_ip, auth_spf: "pass"|"fail"|"softfail"|"none", auth_dkim: "pass"|"fail"|"none", auth_dmarc: "pass"|"fail"|"none", auth_arc, mailer_agent, in_reply_to, references_list: string[], sent_timestamp: int64, content_charset, precedence, list_id, hop_count: int, received_chain: [{from, by, timestamp}], vendor_security_tags: json, headers_json: json }

analysis.attachments (3 partitions · 24h · Key: email_id):
{ email_id, org_id, attachments: [{ sha256, md5, sha1, filename, content_type, content_id, disposition: "inline"|"attachment", size_bytes: int64, entropy: float64, detected_type: string (magic-byte) }] }

analysis.text (6 partitions · 24h · Key: email_id):

{ email_id, org_id, subject, sender_name, sender_domain, plain_text: string (HTML-stripped body), body_language: string (detected), word_count: int }

analysis.plans (6 partitions · 24h · Key: email_id):
{ email_id, org_id, expected_scores: ["url","header","attachment","nlp"] (subset based on email contents), url_count: int, attachment_count: int, has_body: bool, created_at: timestamp }

3a

URL ANALYSIS SERVICE (Go + Python subprocess) — *SVC-03*
Consumes analysis.urls. For each URL in the message:
(i) Redis cache check: key url_score:{sha256(url)} TTL 6h → if hit, reuse score, skip remaining steps.
(ii) TI blocklist: lookup URL domain against enriched_threats table (indexed on url, domain). If exact match with risk_score ≥ 80 → emit score 95, ti_matched=true, skip ML.
(iii) Enrichment (concurrency limit: 5 goroutines): WHOIS (registrar, creation_date, expiry_date, name_servers), SSL cert (issuer, subject, valid_from, valid_to, self-signed check), DNS (A/MX/NS/CNAME), HTTP probe (status_code, redirect_count, final_url, response_time_ms), IP geo (country, asn, asn_name, isp, city).
(iv) ML inference: Enriched features → XGBoost model via Python subprocess (JSON stdin/stdout). Timeout: 5s. On timeout: score=50.
Per-email URL score = max(individual URL scores). Each URL gets its own entry in enrichment tables.

→ **scores.url**
Key: email_id · 6 partitions · 24h retention

R enriched_threats (TI lookup)
R Redis: url_score:{hash}
W enriched_threats (new URLs)
W enrichment_results (raw responses)
W enrichment_jobs (job tracking)
W Redis: url_score:{hash}

Message schema — scores.url:
{ email_id, org_id, component: "url", score: int 0-100, url_count: int, ti_blocked_count: int, ml_scored_count: int, cache_hit_count: int, riskiest_url: string, url_details: [{ url, domain, ti_matched: bool, ti_source: string|null, domain_age_days: int|null, has_ssl: bool, ssl_issuer: string|null, redirect_count: int, ml_score: int|null, is_shortened: bool, enrichment_available: bool }], processing_time_ms: int }

3b

HEADER ANALYSIS SERVICE (Go) — *SVC-04*
Consumes analysis.headers. Produces header_risk_score (0–100) from three analysis dimensions:
(i) Authentication verification: SPF, DKIM, DMARC, ARC result evaluation. From/Reply-To/Return-Path mismatch detection.
(ii) Sender reputation: Domain age analysis (via WHOIS), TI matches on sender domain and originating IP, typosquatting detection (Damerau-Levenshtein distance to known brands), free-provider contextual signals.
(iii) Structural anomaly detection: HTML-only messages, hidden text elements, embedded forms, encoding anomalies, excessive received hops, timestamp drift.
Note: Signal categories are architecturally defined. Specific scoring weights and thresholds are preliminary heuristics subject to empirical calibration.
All signals implemented as configurable rules in the rules table. Each signal that fires creates a rule_hits row for auditability.

→ **scores.header**
Key: email_id · 6 partitions · 24h retention

R rules (active header rules)
R enriched_threats (domain/IP rep.)
W rule_hits (one per fired rule)

Message schema — scores.header:
{ email_id, org_id, component: "header", score: int 0-100, auth_sub_score: int, reputation_sub_score: int, structural_sub_score: int, fired_rules: [{ rule_id, rule_name, rule_version, score_impact: int, match_detail: json }], signals: { spf_result, dkim_result, dmarc_result, from_reply_to_match: bool, from_return_path_match: bool, domain_age_days: int|null, typosquat_target: string|null, typosquat_distance: int|null, is_free_provider: bool, hop_count: int, time_drift_hours: float }, processing_time_ms: int }

3c

ATTACHMENT ANALYSIS SERVICE (Go) — *SVC-05*
Consumes analysis.attachments. For each attachment:
(i) Hash lookup: attachment_library.sha256 → if is_malicious=true, score=90.
(ii) External TI check (if VirusTotal API key configured): query hash → cache in enrichment_results provider="virustotal". TTL: 24h.
(iii) Heuristics: entropy >7.5=+20. Extension vs MIME mismatch (e.g. .pdf with application/x-msdownload)=+30. Dangerous extensions (.exe,.scr,.bat,.ps1,.vbs,.js,.wsf,.hta,.cmd)=+25. Macro-enabled Office (.docm,.xlsm,.pptm)=+20. Double extension (.pdf.exe)=+35.
Per-email attachment score = max(individual attachment scores).

→ **scores.attachment**
Key: email_id · 3 partitions · 24h retention

R attachment_library (hash lookup)
R/W enrichment_results (VT cache)
W attachment_library (update is_malicious, risk_score, threat_tags)

Message schema — scores.attachment:
{ email_id, org_id, component: "attachment", score: int 0-100, attachment_count: int, malicious_count: int, attachment_details:

[{ sha256, filename, content_type, size_bytes, entropy: float, is_malicious: bool, ti_source: string|null, extension_mismatch: bool, has_macros: bool, is_dangerous_extension: bool, individual_score: int }], processing_time_ms: int }

3d

NLP ANALYSIS SERVICE (Python FastAPI + ONNX Runtime) — *SVC-06*

Consumes analysis.text. Produces content_risk_score (0–100) and structured facet outputs from 4 analysis dimensions:

(i) Urgency detection: Identifies false-urgency language patterns. Output: urgency_score 0.0–1.0.

(ii) Intent classification: Classifies email goal. Multi-class: credential_harvesting, malware_delivery, bec, scam, legitimate. Output: intent_label + confidence.

(iii) Brand impersonation: Detects brand impersonation through NER + sender identity analysis. Output: impersonation_score 0.0–1.0.

(iv) Deception language: Identifies manipulative linguistic patterns. Output: deception_score 0.0–1.0.

Note: Analysis facets are architecturally defined. Model architectures (candidate: DistilBERT, TinyBERT), training methodology, and score composition method are under active research.

Individual facet scores are always preserved alongside any composite score for transparency.

Timeout: 10s. On timeout: score=50. Stateless — no DB access.

→ **scores.nlp** Key: email_id · 6 partitions · 24h retention	No database access. Stateless inference service.

Message schema — scores.nlp:

{ email_id, org_id, component: "nlp", score: int 0-100, facets: { urgency_score: float, intent_label: "credential_harvesting"|"malware_delivery"|"bec"|"scam"|"legitimate", intent_confidence: float, impersonation_score: float, impersonated_brand: string|null, deception_score: float }, model_versions: { urgency: string, intent: string, impersonation: string, deception: string }, processing_time_ms: int }

4

SCORE AGGREGATOR SERVICE (Go + Redis state) — *SVC-07*

Consumes 5 topics: analysis.plans + scores.url + scores.header + scores.attachment + scores.nlp.

Maintains per-email state in Redis: Key: aggregator:{email_id} — TTL 120s.

Design: Stateless workers with shared Redis state. Events for the same email may be processed by different instances. Redis is the single source of truth for aggregation state, not instance-local memory.

Function: Gather/synchronization barrier only — does NOT perform scoring. Collects all component scores and packages them into a single payload for the Decision Engine.

Completion check: After each score received, compare received keys to expected list. All present → package and publish to emails.scored. Not all → wait.

Timeout: 30s → emit partial result with partial_analysis: true and list of missing components.

→ **emails.scored** Key: email_id · 6 partitions · 48h retention	**R/W** Redis only: aggregator:{email_id} No PostgreSQL access.

Message schema — emails.scored:

{ email_id, org_id, internal_id: bigint, fetched_at: timestamp, url_score: int|null, header_score: int, attachment_score: int|null, nlp_score: int|null, partial_analysis: bool, missing_components: string[], component_details: { url: <full scores.url message>, header: <full scores.header message>, attachment: <full scores.attachment message>|null, nlp: <full scores.nlp message>|null }, aggregation_latency_ms: int, timeout_triggered: bool }

5

DECISION ENGINE / VERDICT SERVICE (Go) — *SVC-08*

Consumes emails.scored. Terminal processing block — responsible for final scoring, verdict, campaign management, and persistence:

(i) Final score computation: Receives individual component scores (url, header, attachment, nlp) from Aggregator. Combines into a single risk_score (0–100). *Combination method is under research — must be explainable; individual component scores always preserved.*

(ii) Rule engine: Load active rules from rules table (60s cache). Each firing rule adjusts score by score_impact (-100 to +100).

(iii) Verdict: Map final score to label. Preliminary thresholds (subject to calibration): 0–25 → benign | 26–50 → suspicious | 51–75 → phishing | 76–100 → phishing (high) or malware. Confidence computed separately from risk score — reflects label certainty, not severity.

(iv) Campaign management: fingerprint = SHA256(sender_domain|url_domain|subject_template|intent). UPSERT into campaigns. SimHash near-duplicate detection (per-org scoped via Redis).

(v) Campaign-informed scoring: Campaign history (avg risk_score, email count, analyst verdicts) serves as an additional signal for future emails matching the same campaign. *Specific mechanism under research.*

(vi) All DB writes within a single PostgreSQL transaction.

→ **emails.verdict** Key: email_id · 6 partitions · 7d retention	**R** rules (003+008) · **R** campaigns (001+002) · **R** verdicts (campaign history) · **R** Redis: simhash:{org_id}:* **W** emails UPDATE: risk_score, header_risk_score, content_risk_score, url_risk_score, attachment_risk_score, campaign_id, analysis_metadata **W** verdicts INSERT: entity_type, entity_id, label, confidence, source, model_version **W** rule_hits: rule_id, entity_id, score_impact, match_detail **U** campaigns: fingerprint, org_id, threat_type, target_brand, risk_score **W** Redis: simhash:{org_id}:{campaign_id}

Message schema — emails.verdict:
{ email_id, org_id, verdict_label: "benign"|"suspicious"|"phishing"|"malware"|"spam"|"unknown", confidence: float 0-1, risk_score: int 0-100, header_risk_score: int, content_risk_score: int, url_risk_score: int, attachment_risk_score: int, campaign_id: int|null, campaign_fingerprint: string|null, is_new_campaign: bool, fired_rules: [{ rule_id, rule_name, score_impact }], verdict_source: "model"|"rule", model_version: string, partial_analysis: bool, processing_time_total_ms: int }

6a	**NOTIFICATION SERVICE** (Go) — *SVC-09* Consumes emails.verdict. If risk_score ≥ threshold OR verdict_label ∈ {phishing, malware} → send alerts. Channels: email to org admins, webhook POST (SIEM/SOAR), Slack/Teams. **Rate limit:** max 1 alert per campaign_id per org per hour (Redis counter: notif:{org_id}:{campaign_id} TTL 3600s).	
Terminal consumer No output topic.	**R** organisations (thresholds, prefs) · **R** users (admin contacts) **R/W** Redis: rate limit counters	

6b	**API / DASHBOARD SERVICE** (Go REST + React SPA + WebSocket) — *SVC-10* Consumes emails.verdict internally for WebSocket live feed. REST API: /api/v1/emails, /campaigns, /rules (CRUD), /api-keys (CRUD), /stats (MVs), /feeds, /verdicts (analyst POST). Auth: JWT + API keys. Scoped per org via org_id.
Terminal consumer No output topic.	**R** all 17 tables + 5 MVs + 1 view **W** users, organisations, api_keys, rules, verdicts (analyst), audit_log

BG	**TI FEED SYNC SERVICE** (Go) — *SVC-11* — **Independent background service, not in main pipeline.** Runs on schedule (default: every 6h, configurable per feed). For each enabled feed: fetch → parse (CSV/JSON/STIX) → deduplicate → bulk UPSERT into enriched_threats. After all feeds: REFRESH MATERIALIZED VIEW CONCURRENTLY for all 5 views. Update Redis TI cache. **Feeds:** PhishTank, OpenPhish, URLhaus, MalwareBazaar (hashes → attachment_library), Abuse.ch ThreatFox, custom feeds.
No Kafka DB-driven schedule.	**R/W** feeds · **W** enriched_threats (bulk UPSERT) · **W** attachment_library (malware hashes) **W** Redis: TI cache refresh · Triggers MV refresh

2. SERVICE DETAIL SHEETS

2.1 Email Ingestion Service (SVC-01) — Adapter Specifications

Adapter	Protocol	Authentication	Delivery Mechanism	Polling Interval / Trigger	Notes
Gmail	HTTPS (Gmail API v1)	OAuth2 (offline access, scope: gmail.readonly)	Google Pub/Sub push notification → poll messages.get	Event-driven (Pub/Sub) + 5-min fallback poll	Requires GCP project. Watch request expires every 7 days — auto-renew.
Outlook	HTTPS (Microsoft Graph API v1.0)	OAuth2 (app-level or delegated, scope: Mail.Read)	Change notification subscription (webhook)	Event-driven (webhook) + 5-min fallback poll (delta query)	Subscription max lifetime 3 days — auto-renew. Supports shared mailboxes.
IMAP	IMAP4rev1 (TLS)	Username/password or OAuth2 XOAUTH2	IDLE command (push) or poll INBOX	IDLE (instant) + 29-min IDLE timeout reconnect	Generic — works with any IMAP provider. Least metadata available.
API Upload	HTTPS (REST)	API key (header: X-API-Key)	POST /api/v1/scan — multipart/form-data or raw EML body	On-demand (user-initiated)	Primary interface for testing and SIEM forwarding. Rate limit: 100 req/min/org.
Custom	Implements `EmailSource` interface	User-defined	User-defined	User-defined	Interface contract: `Fetch() → []RawEmail`, `Acknowledge(id)`, `HealthCheck() → error`

```
EmailSource interface (Go):
type EmailSource interface {
  Name() string
  Fetch(ctx context.Context, since time.Time) ([]RawEmail, error)
  Acknowledge(ctx context.Context, ids []string) error
  HealthCheck(ctx context.Context) error
}
type RawEmail struct { MessageID string; RFC822 []byte; ReceivedAt time.Time; SourceMetadata json.RawMessage }
```

3. KAFKA TOPIC REGISTRY

Topic	Partitions	Retention	Key	Producer(s)	Consumer Group(s)	Message Avg Size	Expected Throughput
`emails.raw`	6	48h	email_id	SVC-01 (Ingestion)	cg-parser	50–500 KB	~1 msg/s (steady), burst to 50/s
`analysis.urls`	6	24h	email_id	SVC-02 (Parser)	cg-url-analysis	1–10 KB	= emails.raw rate
`analysis.headers`	6	24h	email_id	SVC-02 (Parser)	cg-header-analysis	2–5 KB	= emails.raw rate
`analysis.attachments`	3	24h	email_id	SVC-02 (Parser)	cg-attachment-analysis	0.5–2 KB	~60% of emails.raw (not all emails have attachments)
`analysis.text`	6	24h	email_id	SVC-02 (Parser)	cg-nlp-analysis	1–20 KB	~95% of emails.raw (nearly all have body text)
`analysis.plans`	6	24h	email_id	SVC-02 (Parser)	cg-aggregator	0.2 KB	= emails.raw rate
`scores.url`	6	24h	email_id	SVC-03 (URL Analysis)	cg-aggregator	1–5 KB	= analysis.urls rate
`scores.header`	6	24h	email_id	SVC-04 (Header Analysis)	cg-aggregator	1–3 KB	= analysis.headers rate
`scores.attachment`	3	24h	email_id	SVC-05 (Attachment Analysis)	cg-aggregator	0.5–2 KB	= analysis.attachments rate
`scores.nlp`	6	24h	email_id	SVC-06 (NLP Analysis)	cg-aggregator	0.5–1 KB	= analysis.text rate
`emails.scored`	6	48h	email_id	SVC-07 (Aggregator)	cg-decision-engine	5–30 KB (contains all component details)	= emails.raw rate

`emails.verdict`	6	7d	email_id	SVC-08 (Decision Engine)	cg-notification, cg-dashboard	1–3 KB	= emails.raw rate

Partition key invariant: All topics use email_id as the partition key. This guarantees that all messages related to the same email land on the same partition *within each topic*, enabling ordered processing per email within each consumer. Consumer group cg-aggregator consumes 5 topics — however, cross-topic co-location is NOT guaranteed (topics have different partition counts). This is by design: aggregator instances are stateless workers with shared Redis state. Any instance can process any score event for any email.

4. DATABASE ACCESS MATRIX — COMPLETE

R = Read. **W** = Write (INSERT or UPDATE). **U** = UPSERT. **—** = No access.

Table	SVC-01 Ingestion	SVC-02 Parser	SVC-03 URL Analysis	SVC-04 Header	SVC-05 Attachment	SVC-06 NLP	SVC-07 Aggregator	SVC-08 Decision	SVC-09 Notif.	SVC-10 API/Dash	SVC-11 TI Sync
`emails`	—	**W**	—	—	—	—	—	**W** (UPDATE)	—	R	—
`email_urls`	—	**W**	R	—	—	—	—	—	—	R	—
`email_attachments`	—	**W**	—	—	R	—	—	—	—	R	—
`email_recipients`	—	**W**	—	—	—	—	—	—	—	R	—
`attachment_library`	—	**U**	—	—	R/**W**	—	—	—	—	R	**W**
`enriched_threats`	—	—	R/**W**	R	—	—	—	—	—	R	**U**
`enrichment_results`	—	—	**U**	—	R/**U**	—	—	—	—	R	—
`enrichment_jobs`	—	—	**W**	—	—	—	—	—	—	R	—
`verdicts`	—	—	—	—	—	—	—	R/**W**	—	R/**W**	—
`campaigns`	—	—	—	—	—	—	—	R/**U**	—	R	—
`rules`	—	—	—	R	—	—	—	R	—	R/**W**	—
`rule_hits`	—	—	—	**W**	—	—	—	**W**	—	R	—
`feeds`	—	—	—	—	—	—	—	—	—	R/**W**	R/**W**
`organisations`	R	—	—	—	—	—	—	—	R	R/**W**	—
`users`	—	—	—	—	—	—	—	—	R	R/**W**	—
`api_keys`	R	—	—	—	—	—	—	—	—	R/**W**	—
`audit_log`	—	—	—	—	—	—	—	—	—	**W**	—
mv_threat_summary	—	—	—	—	—	—	—	—	—	R	REFRESH
mv_campaign_summary	—	—	—	—	—	—	—	—	—	R	REFRESH
mv_feed_health	—	—	—	—	—	—	—	—	—	R	REFRESH
mv_rule_performance	—	—	—	—	—	—	—	—	—	R	REFRESH
mv_org_ingestion_summary	—	—	—	—	—	—	—	—	—	R	REFRESH

5. REDIS KEY REGISTRY

Key Pattern	Type	TTL	Set By	Read By	Purpose
`dedup:{org_id}:{message_id}`	STRING (1)	7 days	SVC-01	SVC-01	Email ingestion deduplication. Value: "1". Existence check only.
`url_score:{sha256(url)}`	HASH	6 hours	SVC-03	SVC-03	URL score cache. Fields: score, ti_matched, domain, cached_at. Avoids re-enrichment.
`ti_domain:{domain}`	HASH	1 hour	SVC-11	SVC-03, SVC-04	TI domain cache. Fields: threat_id, risk_score, threat_type. Avoids DB query per URL.
`ti_hash:{sha256}`	HASH	1 hour	SVC-11	SVC-05	TI attachment hash cache. Fields: is_malicious, risk_score, source.
`aggregator:{email_id}`	HASH	120 seconds	SVC-07	SVC-07	Score aggregation state. Fields: expected (JSON), received (JSON), created_at.
`simhash:{org_id}:{campaign_id}`	STRING	30 days	SVC-08	SVC-08	64-bit SimHash for campaign near-duplicate detection. Per-org scoped to prevent cross-tenant matching.
`notif:{org_id}:{campaign_id}`	STRING (counter)	3600 seconds	SVC-09	SVC-09	Notification rate limiter. INCR on each alert. Block if > 1.
`rules_cache:{org_id}`	STRING (JSON)	60 seconds	SVC-04, SVC-08	SVC-04, SVC-08	In-memory rule cache. Full rule set serialized. Avoids DB poll per email.

6. ERROR HANDLING & TIMEOUT POLICY

Service	Failure Scenario	Behavior	Retry Policy	Fallback Score

SVC-01	Adapter fetch failure	Log error, retry on next poll cycle	Exponential backoff, max 3 retries per cycle	N/A (no score)
SVC-01	Kafka publish failure	Retry with backoff, dead-letter after 5 attempts	1s, 2s, 4s, 8s, 16s	N/A
SVC-02	MIME parse failure	Log error, publish with empty URLs/attachments, body_plain = raw text extraction attempt	No retry (deterministic failure)	N/A
SVC-03	WHOIS/SSL/DNS timeout (per URL)	Skip enrichment for that URL, use cached data or empty enrichment	3 retries, 2s timeout each	score = 50
SVC-03	XGBoost subprocess timeout (5s)	Return neutral score, log model timeout	No retry (latency-critical)	score = 50
SVC-03	XGBoost subprocess crash	Respawn process from pool, retry once	1 retry, then score = 50	score = 50
SVC-04	Rule evaluation error	Skip malformed rule, log, continue with remaining rules	No retry (skip rule)	Rule impact = 0
SVC-05	VirusTotal API rate limit (429)	Use local hash lookup only, skip VT enrichment	Backoff until next rate window	Use local-only score
SVC-06	NLP inference timeout (10s)	Return neutral score, flag partial_analysis	No retry (latency-critical)	score = 50
SVC-06	Model loading failure (OOM)	Service returns 503, consumer pauses	Kubernetes restart + health check	score = 50 (via aggregator timeout)
SVC-07	Score never arrives (component down)	30s timeout → emit partial result with missing = 50	No retry (timeout-based)	missing = 50
SVC-07	Redis connection failure	In-memory fallback (non-durable), log critical	Reconnect with backoff	Functions degraded
SVC-08	DB write failure (tx rollback)	Retry entire transaction	3 retries, exponential backoff	Kafka offset not committed → reprocessed
SVC-11	Feed fetch failure (HTTP error)	Log error, skip feed, continue with others	Will retry next 6h cycle	Stale data (last successful sync)

7. DEDUPLICATION STRATEGY — COMPLETE

Stage	What Is Deduped	Method	Storage	Scope
Email Ingestion (SVC-01)	Duplicate email deliveries	Check `(org_id, message_id)` existence	Redis set (fast path) → DB unique constraint (authoritative)	Per-org (Message-IDs not globally unique)
URL Analysis (SVC-03)	Recently-analyzed URLs	SHA256 of full URL → Redis cache lookup	Redis: `url_score:{hash}` TTL 6h	Global (same URL across all orgs)
Attachment Library	Identical file binaries	SHA256 unique constraint on `attachment_library`	PostgreSQL unique index	Global (cross-tenant — intentional for threat signal)
TI Feed Sync (SVC-11)	Duplicate threat indicators	URL unique constraint on `enriched_threats`	PostgreSQL unique index	Global
Campaign Detection (SVC-08)	Near-duplicate emails (same campaign)	SimHash of body text, Hamming distance ≤ 3	Redis: `simhash:{org_id}:{campaign_id}` TTL 30d	Per-org
Campaign Fingerprinting (SVC-08)	Exact-match campaign grouping	SHA256 of normalized (sender_domain + url_domain + subject_template + intent)	PostgreSQL: `campaigns.fingerprint` UNIQUE	Per-org (after org_id addition)
Notification (SVC-09)	Alert fatigue — repeated alerts for same campaign	Redis counter per (org_id, campaign_id) with 1h TTL	Redis: `notif:{org_id}:{campaign_id}`	Per-org per-campaign

8. CAMPAIGN DETECTION ALGORITHM

8.1 Campaign Fingerprint Computation (Exact Match)

```
INPUT: scored email with verdict

  sender_domain  = lower(email.sender_domain)                          // e.g. "paypa1.com"
  url_domain     = lower(domain_of(email.riskiest_url))                // e.g. "secure-login.paypa1.com"
                   OR "" if no URLs
  subject_tpl    = template(email.subject)                             // replace names/nums with placeholders
                   e.g. "Your {BRAND} account #{NUM} needs verification"
  intent         = nlp_facets.intent_label                             // e.g. "credential_harvesting"

  fingerprint    = SHA256( sender_domain + "|" + url_domain + "|" + SHA256(subject_tpl) + "|" + intent )

DB OPERATION:
  INSERT INTO campaigns (fingerprint, org_id, threat_type, target_brand, first_seen, last_seen, risk_score)
  VALUES ($fingerprint, $org_id, $threat_type, $target_brand, NOW(), NOW(), $risk_score)
  ON CONFLICT (fingerprint) DO UPDATE SET
      last_seen = NOW(),
      risk_score = (campaigns.risk_score + $risk_score) / 2;   -- rolling average

  UPDATE emails SET campaign_id = <returned campaign_id> WHERE internal_id = $email_id AND fetched_at = $fetched_at;
```

8.2 SimHash Near-Duplicate Detection (Fuzzy Match)

```
INPUT: email plain_text body

  simhash = compute_simhash_64bit(tokenize(plain_text))       // 64-bit SimHash

  FOR each campaign_id in Redis keys matching simhash:{org_id}:*:
      stored_hash = GET simhash:{org_id}:{campaign_id}
      distance = hamming_distance(simhash, stored_hash)
      IF distance <= 3:                                        // threshold: 3 bits out of 64
          → associate email with this campaign (even if fingerprint differs)
          → BREAK (take first match; campaigns ordered by last_seen DESC)

  IF no match found AND email.risk_score >= 50:
      → new campaign was already created by fingerprint logic above
      → SET simhash:{org_id}:{campaign_id} = simhash (TTL 30d)
```

9. DATA PRIVACY & RETENTION POLICY

Data Category	Storage Location	Retention	Purge Method	Justification
Raw email body (body_plain, body_html)	PostgreSQL `emails`	Configurable per org (default: 30 days)	Scheduled job: `UPDATE emails SET body_plain=NULL, body_html=NULL WHERE fetched_at < NOW() - retention_interval AND org_id = $org`	Contains user content / PII. Not needed after analysis.
Email metadata (sender, subject, headers, scores)	PostgreSQL `emails`	Indefinite	Soft delete via `deleted_at` (user request)	Required for campaign tracking, trend analysis, dashboards.
Raw email bytes	Kafka `emails.raw`	48 hours	Kafka topic retention policy (automatic)	Needed only for replay/reprocessing. Auto-expires.
Analysis intermediate messages	Kafka `analysis.*`, `scores.*`	24 hours	Kafka topic retention policy (automatic)	Transient pipeline data. Not needed after verdict.
Verdict messages	Kafka `emails.verdict`	7 days	Kafka topic retention policy (automatic)	Longer retention for notification retry / dashboard catch-up.
Verdicts (DB)	PostgreSQL `verdicts`	Indefinite (append-only)	Never deleted — compliance audit trail	Immutable record of all classification decisions.
Attachment binaries	Object store (S3 / MinIO via `storage_uri`)	90 days	S3 lifecycle policy	Useful for forensic analysis. Hash + metadata persist indefinitely.
Attachment hashes & metadata	PostgreSQL `attachment_library`	Indefinite	N/A	Hash-only — no PII. Cross-tenant malware signal.
Threat intelligence	PostgreSQL `enriched_threats`	Indefinite (soft-delete stale)	`deleted_at` set for entries not seen in 90 days	TI data is valuable for historical correlation.
Enrichment API responses	PostgreSQL `enrichment_results`	TTL-based (`expires_at`)	Staleness sweep re-fetches expired entries	Cacheable external API responses. Re-fetch when stale.
Audit log	PostgreSQL `audit_log`	Indefinite (append-only)	Never deleted — compliance	Regulatory / compliance requirement.

10. INFRASTRUCTURE REQUIREMENTS

PostgreSQL 15+

- 16 tables, 5 materialized views, 1 view
- Monthly range partitioning on emails
- Row-level security per org_id (migration 018)
- JSONB + GIN indexes for headers, tags
- Append-only verdicts (no UPDATE/DELETE)
- pg_cron for MV refresh schedule
- Recommended: 4 vCPU, 16 GB RAM, SSD
- Connection pool: pgbouncer (transaction mode)

Apache Kafka

- 12 topics (see §3 for full registry)
- Minimum 3 brokers (production)
- Replication factor: 3 (ISR min: 2)
- Partition key: email_id on all topics
- Consumer groups: 1 per consuming service
- Exactly-once semantics (idempotent producers + transactional consumers)
- Recommended: 2 vCPU, 8 GB RAM per broker
- ZooKeeper or KRaft mode (KRaft preferred)

Redis 7+

- 8 key patterns (see §5 for full registry)
- Persistence: AOF with 1s fsync
- Max memory: 2 GB (LRU eviction on TI cache keys)
- Used for: dedup, URL score cache, TI cache, aggregator state, rate limiting, SimHash store, rule cache
- Single instance sufficient; Sentinel for HA
- Recommended: 2 vCPU, 4 GB RAM

11. SCALING STRATEGY

Service	Stateless?	Scaling Method	Scale Trigger	Min / Max Instances
SVC-01 Ingestion	Yes	Horizontal (multiple adapter instances)	Ingestion lag > 1000 emails	1 / 4
SVC-02 Parser	Yes	Horizontal (Kafka consumer group)	Consumer lag on `emails.raw` > 500	1 / 6
SVC-03 URL Analysis	Yes	Horizontal (Kafka consumer group)	Consumer lag on `analysis.urls` > 500	1 / 6
SVC-04 Header Analysis	Yes	Horizontal (Kafka consumer group)	Consumer lag on `analysis.headers` > 1000	1 / 3
SVC-05 Attachment Analysis	Yes	Horizontal (Kafka consumer group)	Consumer lag on `analysis.attachments` > 500	1 / 3
SVC-06 NLP Analysis	Yes	Horizontal (Kafka consumer group)	Consumer lag on `analysis.text` > 200 (heavier compute)	1 / 4
SVC-07 Aggregator	No (Redis state)	Horizontal OK (shared Redis state)	Consumer lag on any `scores.*` > 500	1 / 3
SVC-08 Decision Engine	Yes	Horizontal (Kafka consumer group)	Consumer lag on `emails.scored` > 500	1 / 3
SVC-09 Notification	Yes	Horizontal	Rarely bottleneck	1 / 2
SVC-10 API/Dashboard	Yes	Horizontal (behind load balancer)	HTTP request latency p95 > 500ms	2 / 6
SVC-11 TI Sync	Yes (scheduled)	Single instance (leader election)	N/A	1 / 1

12. OBSERVABILITY & SLA TARGETS

Metric	Target	Measurement Point	Alert Threshold
End-to-end latency (ingestion → verdict)	< 60s p95	`emails.verdict.processing_time_total_ms`	> 90s p95 over 5 min window
Email Parser throughput	> 10 emails/s sustained	Kafka consumer lag on `emails.raw`	Lag > 1000 for > 5 min
ML inference latency (URL XGBoost)	< 5s p99	SVC-03 histogram	> 5s p99 (timeout territory)
ML inference latency (NLP ensemble)	< 10s p99	SVC-06 histogram	> 10s p99 (timeout territory)
Score Aggregator completeness	> 95% full analysis (not partial)	`partial_analysis: true` rate	> 10% partial over 15 min
TI feed freshness	< 7 hours since last sync	`feeds.last_fetched_at`	> 8 hours since last successful sync
DB connection pool utilization	< 80%	pgbouncer stats	> 90% for > 2 min
Redis memory utilization	< 80% of maxmemory	Redis INFO memory	> 90%

Kafka disk usage per broker	< 70%	Broker filesystem	> 85%
Error rate (any service)	< 1% of processed messages	Per-service error counters	> 5% over 5 min

Tracing: OpenTelemetry SDK in all Go/Python services. Trace context propagated via Kafka headers (traceparent). Traces exported to Jaeger. email_id attached as span attribute on every operation for cross-service correlation.
Logging: zerolog (Go), structlog (Python). JSON format. Fields: service, email_id, org_id, trace_id, level, msg, error. Shipped to ELK or Loki.
Metrics: Prometheus exposition on :9090/metrics per service. Grafana dashboards.

13. TABLE ORIGIN REFERENCE — MIGRATION FILE → TABLE

Table / View	Defined In	Columns Added In	Column Count
`enriched_threats`	001_initial_schema.up.sql	002 (+org_id), 003 (+feed_id)	35
`campaigns`	001_initial_schema.up.sql	002 (+org_id)	13
`emails` (partitioned)	001_initial_schema.up.sql	002 (+org_id, +uq_emails_org_message_id)	30
`verdicts`	001_initial_schema.up.sql	002 (+created_by)	10
`email_urls`	001_initial_schema.up.sql	—	5
`attachment_library`	001_initial_schema.up.sql	008 (+storage_uri, +updated_at)	13
`email_attachments`	001_initial_schema.up.sql	008 (+created_at)	10
`organisations`	002_add_users_orgs.up.sql	—	6
`users`	002_add_users_orgs.up.sql	—	10
`api_keys`	002_add_users_orgs.up.sql	—	10
`audit_log`	002_add_users_orgs.up.sql	—	9
`feeds`	003_add_enrichments.up.sql	—	9
`enrichment_jobs`	003_add_enrichments.up.sql	—	10
`enrichment_results`	003_add_enrichments.up.sql	008 (+ttl_seconds, +expires_at)	11
`rules`	003_add_enrichments.up.sql	008 (+rule_group_id)	11
`rule_hits`	003_add_enrichments.up.sql	—	7
`email_recipients`	008_add_missing_schema.up.sql	—	7
`current_verdicts` (VIEW)	001_initial_schema.up.sql	—	8 (DISTINCT ON verdicts)
`mv_threat_summary`	004_add_materialized_view.up.sql	—	Aggregates enriched_threats
`mv_campaign_summary`	004_add_materialized_view.up.sql	—	Joins campaigns + emails + verdicts
`mv_feed_health`	004_add_materialized_view.up.sql	—	Joins feeds + enriched_threats
`mv_rule_performance`	004_add_materialized_view.up.sql	—	Joins rules + rule_hits
`mv_org_ingestion_summary`	004_add_materialized_view.up.sql	—	Joins emails + verdicts

Design note — email_urls.threat_id FK dependency:
The email_urls table (001) has a threat_id BIGINT NOT NULL REFERENCES enriched_threats(id). This means the Email Parser (SVC-02) must UPSERT a bare row into enriched_threats (with just url + domain) *before* it can INSERT into email_urls, to obtain the threat_id FK. The URL Analysis Service (SVC-03) later UPDATEs that same enriched_threats row with full enrichment data (WHOIS, SSL, geo, etc.). This two-phase write pattern is documented in the column-level flow below.

14. COLUMN-LEVEL DATA FLOW — FULL TRACEABILITY

For every pipeline step: exactly which table columns are read, which are written, and how Kafka message fields map to DB columns. Migration file references in parentheses.

Step 1 — Email Ingestion Service (SVC-01)

Operation	Table (Migration)	Columns Accessed	Direction	Notes
Auth lookup	`api_keys` (002)	`key_hash`, `org_id`, `scopes`, `revoked_at`, `expires_at`	READ	Validate API key → extract org_id. Also updates `last_used_at`.
Ingestion quota	`organisations` (002)	`id`, `monthly_ingestion_limit`	READ	Enforce per-org monthly ingestion cap. Cached in Redis.
Dedup (fast)	Redis	`dedup:{org_id}:{message_id}`	READ then WRITE	SETNX check. If key exists → duplicate, discard. If not → set with TTL 7d.
Dedup (authoritative)	`emails` (001+002)	`org_id`, `message_id`, `fetched_at`	READ (via constraint)	Unique constraint `uq_emails_org_message_id(org_id, message_id, fetched_at)` from migration 002. Falls through to this only on Redis miss.

Kafka output emails.raw → DB column mapping:
The emails.raw message carries raw_rfc822 bytes. These are NOT written to any DB column at this stage. The raw bytes flow to SVC-02 (Parser) which extracts them into individual emails table columns.

Step 2 — Email Parser Service (SVC-02)

Operation	Table (Migration)	Columns Written	Source
INSERT email	`emails` (001+002)	`internal_id` (auto), `fetched_at` (from Kafka msg), `org_id` (from Kafka msg, 002), `message_id`, `sender_name`, `sender_email`, `sender_domain`, `reply_to_email`, `return_path`, `originating_ip`, `auth_spf`, `auth_dkim`, `auth_dmarc`, `auth_arc`, `x_originating_ip`, `mailer_agent`, `in_reply_to`, `references_list`, `content_charset`, `precedence`, `list_id`, `vendor_security_tags` (JSONB),	All extracted from parsed RFC 822 MIME. Maps 1:1 to `ParsedEmail` struct fields in `pkg/models/email.go`.

		subject, sent_timestamp, headers_json (JSONB), body_plain, body_html	
Score columns NOT written at this stage: header_risk_score, content_risk_score, attachment_risk_score, url_risk_score, risk_score, campaign_id, analysis_metadata — these remain at DEFAULT 0 / NULL until Decision Engine (Step 5) updates them.			
UPSERT bare URL	enriched_threats (001)	url, domain, tld	For each URL extracted from email body/headers. UPSERT by url unique constraint. Writes ONLY url+domain+tld. All enrichment columns (WHOIS, SSL, geo, etc.) left NULL — populated later by SVC-03.
INSERT URL link	email_urls (001)	email_id → emails.internal_id, email_fetched_at → emails.fetched_at, threat_id → enriched_threats.id (from UPSERT above), visible_text	One row per URL per email. threat_id FK obtained from the enriched_threats UPSERT RETURNING id.
UPSERT attachment hash	attachment_library (001+008)	sha256, md5, sha1, actual_extension, size_bytes, entropy	UPSERT by sha256 unique constraint. is_malicious, risk_score, threat_tags, storage_uri left at defaults — updated later by SVC-05 / SVC-11.
INSERT attachment link	email_attachments (001+008)	email_id, email_fetched_at, attachment_id → attachment_library.id, filename, content_type, content_id, disposition, created_at (008)	One row per attachment per email. analysis_metadata and risk_score left NULL/0 — updated by SVC-05.
INSERT recipients	email_recipients (008)	email_id, email_fetched_at, org_id, address, display_name, recipient_type ('to'|'cc'|'bcc')	One row per recipient (To, CC, BCC) extracted from headers.

Kafka output fields → DB column traceability:

analysis.urls fields: url → comes from parser's URL extraction → already written to enriched_threats.url; visible_text → already written to email_urls.visible_text. The Kafka message carries them to SVC-03 for scoring.

analysis.headers fields: sender_email → emails.sender_email; sender_domain → emails.sender_domain; auth_spf → emails.auth_spf; auth_dkim → emails.auth_dkim; auth_dmarc → emails.auth_dmarc; auth_arc → emails.auth_arc; originating_ip → emails.originating_ip; x_originating_ip → emails.x_originating_ip; mailer_agent → emails.mailer_agent; reply_to_email → emails.reply_to_email; return_path → emails.return_path; in_reply_to → emails.in_reply_to; references_list → emails.references_list; sent_timestamp → emails.sent_timestamp; content_charset → emails.content_charset; precedence → emails.precedence; list_id → emails.list_id; vendor_security_tags → emails.vendor_security_tags; headers_json → emails.headers_json. All already persisted in DB by the INSERT above; the Kafka message passes copies to SVC-04 for analysis so SVC-04 doesn't need to re-read the DB.

analysis.attachments fields: sha256 → attachment_library.sha256; md5 → attachment_library.md5; sha1 → attachment_library.sha1; filename → email_attachments.filename; content_type → email_attachments.content_type; size_bytes → attachment_library.size_bytes; entropy → attachment_library.entropy. Already persisted; copies sent to SVC-05.

analysis.text fields: plain_text → emails.body_plain; subject → emails.subject; sender_name → emails.sender_name; sender_domain → emails.sender_domain. Already persisted; copies sent to SVC-06. Fields body_language and word_count are computed by the parser and only exist in the Kafka message (not stored in DB).

Step 3a — URL Analysis Service (SVC-03)

Operation	Table (Migration)	Columns Accessed	Direction	Details
TI blocklist lookup	enriched_threats (001)	id, url, domain, risk_score, threat_type, target_brand, deleted_at	READ	Lookup by url (unique index) or domain (idx_enriched_domain). Only active rows (deleted_at IS NULL). Cached in Redis ti_domain:{domain}.
UPDATE enrichment data	enriched_threats (001)	online, http_status_code, ip_address, cidr_block, asn, asn_name, isp, country, country_name, region, city, latitude, longitude, ssl_enabled, cert_issuer, cert_subject, cert_valid_from, cert_valid_to, cert_serial, tld, registrar, creation_date, expiry_date, updated_date, name_servers, page_language, page_title, last_checked, last_seen, risk_score (from ML), analysis_metadata (JSONB)	WRITE (UPDATE)	Updates the bare row created by SVC-02. All 28 enrichment columns populated from WHOIS, SSL, DNS, HTTP, geo API responses. Maps 1:1 to EnrichmentData struct in pkg/models/url.go.
INSERT enrichment result	enrichment_results (003+008)	entity_type='threat', entity_id → enriched_threats.id, provider (e.g. 'whois','ipinfo','ssllabs'), raw_response (JSONB), malicious_votes, harmless_votes, suspicious_votes, reputation_score, ttl_seconds (008), expires_at (008, trigger-computed)	UPSERT	One row per provider per URL. UPSERT by (entity_type, entity_id, provider) unique constraint. Raw API responses stored for re-scoring without re-fetching.
INSERT enrichment job	enrichment_jobs (003)	job_type (whois|dns|asn|ip_geo|ssl_cert|url_scan), status, entity_type='threat', entity_id, attempts, max_attempts=3, last_error, started_at, completed_at	WRITE	One job per enrichment type per URL. Tracks retries and failures. Partial index

				`idx_jobs_pending` (003) used for worker poll.

Kafka scores.url output → data provenance:
score → computed from XGBoost ML on features extracted from enriched_threats columns (domain_age = NOW()-creation_date, ssl_enabled, registrar, country, etc.).
ti_matched → true if enriched_threats.risk_score ≥ 80 at lookup time.
domain_age_days → NOW() - enriched_threats.creation_date.
has_ssl → enriched_threats.ssl_enabled.
ssl_issuer → enriched_threats.cert_issuer.
redirect_count → from HTTP probe, stored in enrichment_results.raw_response JSONB.
None of these score fields are written to DB at this stage — they flow via Kafka to SVC-07 (Aggregator) then SVC-08 (Decision Engine) which writes the final emails.url_risk_score.

Step 3b — Header Analysis Service (SVC-04)

Operation	Table (Migration)	Columns Read	Details
Load rules	`rules` (003+008)	`id`, `org_id`, `name`, `version`, `status`, `logic` (JSONB), `score_impact`, `target`, `rule_group_id` (008)	WHERE `status='active'` AND (`target='header'` OR `target='email'`) AND (`org_id=?` OR `org_id` IS NULL). Cached in Redis `rules_cache:{org_id}` TTL 60s.
Domain reputation	`enriched_threats` (001)	`domain`, `risk_score`, `threat_type`, `creation_date`	Lookup sender_domain (from Kafka message) against `enriched_threats.domain`. Used for: domain age check (`creation_date`), TI match (`risk_score`). Cached in Redis `ti_domain:{domain}`.

Operation	Table (Migration)	Columns Written	Details
Record rule fires	`rule_hits` (003)	`rule_id` → `rules.id`, `rule_version` → `rules.version`, `entity_type='email'`, `entity_id` → `emails.internal_id`, `score_impact` → `rules.score_impact`, `match_detail` (JSONB: which signal fired, e.g. {"signal":"spf_fail","value":"fail"})	One INSERT per fired rule. Append-only. Links back to both the rule definition and the email it fired on.

Input traceability — Kafka analysis.headers fields → emails columns (001):
Every field in the analysis.headers Kafka message maps to a column already persisted in the emails table by SVC-02. SVC-04 receives them via Kafka to avoid a DB round-trip. The mapping is:
auth_spf ← emails.auth_spf (001) | auth_dkim ← emails.auth_dkim (001) | auth_dmarc ← emails.auth_dmarc (001) | auth_arc ← emails.auth_arc (001) | sender_domain ← emails.sender_domain (001) | reply_to_email ← emails.reply_to_email (001) | return_path ← emails.return_path (001) | originating_ip ← emails.originating_ip (001) | x_originating_ip ← emails.x_originating_ip (001) | mailer_agent ← emails.mailer_agent (001) | sent_timestamp ← emails.sent_timestamp (001) | headers_json ← emails.headers_json (001, JSONB — contains the received chain for hop count analysis)

Output: scores.header carries the computed header_risk_score. This value is NOT written to DB here — it flows to SVC-07→SVC-08 which writes emails.header_risk_score (001).

Step 3c — Attachment Analysis Service (SVC-05)

Operation	Table (Migration)	Columns Accessed	Direction	Details
Hash lookup	`attachment_library` (001+008)	`sha256`, `is_malicious`, `risk_score`, `threat_tags`	READ	Lookup by `sha256` (unique index). Cached in Redis `ti_hash:{sha256}`.
VT enrichment cache	`enrichment_results` (003+008)	`entity_type='attachment'`, `entity_id`, `provider='virustotal'`, `raw_response`, `malicious_votes`, `harmless_votes`, `suspicious_votes`, `expires_at` (008)	READ then UPSERT	Check if cached VT result exists and is not expired (`expires_at` > NOW()). If miss or stale → query VT API → UPSERT with `ttl_seconds=86400` (008).
Update malicious flag	`attachment_library` (001)	`is_malicious`, `risk_score`, `threat_tags`, `updated_at` (008, trigger)	WRITE (UPDATE)	If VT returns malicious_votes > 0 or heuristics flag dangerous: update `is_malicious=true`, `risk_score`, append to `threat_tags`. The `updated_at` trigger (008) fires automatically.

Input traceability — Kafka analysis.attachments fields → DB columns:
sha256 ← attachment_library.sha256 (001) | md5 ← attachment_library.md5 (001) | sha1 ← attachment_library.sha1 (001) | filename ← email_attachments.filename (001) | content_type ← email_attachments.content_type (001) | entropy ← attachment_library.entropy (001) | size_bytes ← attachment_library.size_bytes (001) | content_id ← email_attachments.content_id (001) | disposition ← email_attachments.disposition (001)

Output: scores.attachment carries the computed attachment_risk_score. Flows to SVC-07→SVC-08 which writes emails.attachment_risk_score (001).

Step 3d — NLP Analysis Service (SVC-06)

Operation	Table (Migration)	Columns Accessed	Direction	Details
No database access. Stateless inference only.				

Input traceability — Kafka analysis.text fields → DB columns:
plain_text ← emails.body_plain (001) | subject ← emails.subject (001) | sender_name ← emails.sender_name (001) |

sender_domain ← emails.sender_domain (001)
Fields body_language and word_count are parser-computed and not stored in any DB table.

Output: scores.nlp carries the computed content_risk_score. Flows to SVC-07→SVC-08 which writes emails.content_risk_score (001).

Step 4 — Score Aggregator Service (SVC-07)

Operation	Storage	Details
State tracking	Redis only	Key `aggregator:{email_id}` (TTL 120s). No PostgreSQL tables accessed. Consumes from 5 Kafka topics, produces to 1. Pure event-driven state machine.

Score provenance in emails.scored output:
url_score → from scores.url.score (SVC-03) → destined for emails.url_risk_score (001)
header_score → from scores.header.score (SVC-04) → destined for emails.header_risk_score (001)
attachment_score → from scores.attachment.score (SVC-05) → destined for emails.attachment_risk_score (001)
nlp_score → from scores.nlp.score (SVC-06) → destined for emails.content_risk_score (001)
aggregated_score → weighted combination → destined for emails.risk_score (001) after rule adjustment in SVC-08

Step 5 — Decision Engine / Verdict Service (SVC-08)

Operation	Table (Migration)	Columns Accessed	Direction	SQL Pattern
Load rules	`rules` (003+008)	`id`, `org_id`, `name`, `version`, `status`, `logic`, `score_impact`, `target`, `rule_group_id` (008)	READ	`SELECT * FROM rules WHERE status='active' AND (org_id=? OR org_id IS NULL)`
Campaign lookup	`campaigns` (001+002)	`id`, `fingerprint`, `org_id` (002)	READ	`SELECT id FROM campaigns WHERE fingerprint=?`
UPDATE email scores	`emails` (001)	`risk_score` ← final_score (after rule adjustments), `header_risk_score` ← from scores.header, `content_risk_score` ← from scores.nlp, `url_risk_score` ← from scores.url, `attachment_risk_score` ← from scores.attachment, `campaign_id` ← from campaign UPSERT below, `analysis_metadata` (JSONB) ← full component breakdown	WRITE (UPDATE)	`UPDATE emails SET risk_score=?, header_risk_score=?, content_risk_score=?, url_risk_score=?, attachment_risk_score=?, campaign_id=?, analysis_metadata=? WHERE internal_id=? AND fetched_at=?`
INSERT verdict	`verdicts` (001+002)	`entity_type`='email', `entity_id` → `emails.internal_id`, `label` ← verdict_label ENUM (001), `confidence` ← computed separately: base = distance from nearest threshold / 25, penalties for partial_analysis (×0.7) and rule-only verdict (×0.5), `source` ← 'model' verdict_source ENUM (001), `model_version` ← from config, `notes` ← NULL (automated), `created_by` (002) ← NULL (automated)	WRITE (INSERT)	Append-only. Never UPDATE existing verdicts. Current verdict resolved by `current_verdicts` VIEW (001): `DISTINCT ON (entity_type, entity_id) ORDER BY created_at DESC`.
INSERT rule hits	`rule_hits` (003)	`rule_id`, `rule_version`, `entity_type`='email', `entity_id`, `score_impact`, `match_detail` (JSONB)	WRITE (INSERT)	One row per rule that fired. Same structure as SVC-04 rule_hits but with target='email' rules.
UPSERT campaign	`campaigns` (001+002)	`fingerprint` ← SHA256(sender_domain|url_domain|subject_tpl|intent), `org_id` (002), `name` ← auto-generated, `threat_type`, `target_brand`, `first_seen` / `last_seen`, `risk_score` ← rolling avg	UPSERT	`INSERT INTO campaigns (fingerprint, org_id, ...) ON CONFLICT (fingerprint) DO UPDATE SET last_seen=NOW(), risk_score= (campaigns.risk_score + $score)/2`

Step 6a — Notification Service (SVC-09)

Operation	Table (Migration)	Columns Read	Details
Org preferences	`organisations` (002)	`id`, `name`, `slug`, `monthly_ingestion_limit`	Notification threshold not yet a column — requires migration (see §15 Schema Gap #1). Until migration is applied, SVC-09 uses config-based defaults: threshold=70, channels=["email"]. Migration adds `notification_threshold INT DEFAULT 70` and `notification_channels JSONB DEFAULT '["email"]'` to organisations table.
Admin contacts	`users` (002)	`id`, `org_id`, `email`, `display_name`, `role`	`SELECT email, display_name FROM users WHERE org_id=? AND role='admin' AND deleted_at IS NULL`

Step 6b — API / Dashboard Service (SVC-10)

Operation	Table (Migration)	Columns	Direction
Email list/detail	`emails` (001+002)	All 30 columns	READ
Email URLs	`email_urls` (001) JOIN `enriched_threats` (001)	All columns from both (URL details + threat details)	READ
Email attachments	`email_attachments` (001+008) JOIN `attachment_library` (001+008)	All columns from both	READ

Email recipients	email_recipients (008)	All 7 columns	READ
Current verdict	current_verdicts VIEW (001)	entity_type, entity_id, label, confidence, source, model_version, notes, created_at	READ
Verdict history	verdicts (001+002)	All 10 columns	READ
Campaign list/detail	mv_campaign_summary (004)	All MV columns (joins campaigns+emails+verdicts)	READ
Org dashboard stats	mv_org_ingestion_summary (004)	All MV columns (aggregates emails+verdicts)	READ
TI dashboard	mv_threat_summary (004)	All MV columns (aggregates enriched_threats)	READ
Feed health	mv_feed_health (004)	All MV columns (joins feeds+enriched_threats)	READ
Rule performance	mv_rule_performance (004)	All MV columns (joins rules+rule_hits)	READ
Rule CRUD	rules (003+008)	All 11 columns incl. rule_group_id (008)	READ + WRITE
Analyst verdict	verdicts (001+002)	INSERT: entity_type, entity_id, label, confidence, source='analyst', notes, created_by (002) → users.id	WRITE
User/org mgmt	users (002), organisations (002)	All columns	READ + WRITE
API key mgmt	api_keys (002)	All 10 columns	READ + WRITE
Feed mgmt	feeds (003)	All 9 columns	READ + WRITE
Audit logging	audit_log (002)	org_id, user_id, api_key_id, action, entity_type, entity_id, diff (JSONB), ip_address	WRITE (every mutation)
Enrichment jobs	enrichment_jobs (003)	All 10 columns	READ (monitoring)
Enrichment results	enrichment_results (003+008)	All 11 columns	READ (detail views)
Rule hits	rule_hits (003)	All 7 columns	READ (explainability)

Background — TI Feed Sync Service (SVC-11)

Operation	Table (Migration)	Columns	Direction	Details
Load feed config	feeds (003)	id, name, feed_type, url, last_fetched_at, enabled, reliability_weight	READ	SELECT * FROM feeds WHERE enabled=TRUE
Update feed status	feeds (003)	last_fetched_at, updated_at (trigger)	WRITE	UPDATE feeds SET last_fetched_at=NOW() WHERE id=?
UPSERT threat indicators	enriched_threats (001+002+003)	url, domain, tld, threat_type, target_brand, threat_tags, source_feed (legacy), feed_id (003) → feeds.id, source_id, first_seen, last_seen, risk_score, org_id (002, NULL for global feeds)	UPSERT	INSERT INTO enriched_threats (url, domain, ...) ON CONFLICT (url) DO UPDATE SET last_seen=NOW(), threat_tags=array_cat(enriched_threats.threat_tags, $new_tags), risk_score=GREATEST(enriched_threats.risk_score, $score). Enrichment columns (WHOIS/SSL/geo) NOT written by TI sync — only by SVC-03.
UPSERT malware hashes	attachment_library (001)	sha256, is_malicious=TRUE, risk_score, threat_tags	UPSERT	MalwareBazaar feed provides SHA256 hashes of known malware. INSERT INTO attachment_library (sha256, is_malicious, risk_score, threat_tags) ON CONFLICT (sha256) DO UPDATE SET is_malicious=TRUE, risk_score=GREATEST(...)
Refresh MVs	All 5 materialized views (004)	—	REFRESH	Calls refresh_all_materialized_views() (function defined in 004). CONCURRENTLY mode, no read blocking.
Update Redis caches	Redis	ti_domain:{domain}, ti_hash:{sha256}	WRITE	Bulk-load recently-updated enriched_threats domains and attachment_library hashes into Redis for fast lookup by SVC-03/04/05.

15. IMPLEMENTATION DELTAS — TRACKED GAPS

The following items are known differences between the architecture as specified and the current schema/implementation state. These are tracked deltas, not unresolved design questions. Each has a clear resolution path.

Gap	Affected Table	Migration Needed?	Resolution
organisations lacks notification config columns	organisations (002)	Yes — new migration	Add notification_threshold INT DEFAULT 70, notification_channels JSONB DEFAULT '[]', retention_days INT DEFAULT 30. SVC-09 reads these; SVC-10 writes them.
email_attachments.risk_score never updated by SVC-05	email_attachments (001)	No — just implementation	SVC-05 should UPDATE email_attachments SET risk_score=?, analysis_metadata=? WHERE

			`email_id=? AND attachment_id=?` after scoring each attachment.
`enrichment_results.suspicious_votes` column present but not referenced	`enrichment_results` (003)	No	SVC-03 and SVC-05 should populate this from VirusTotal `last_analysis_stats.suspicious` field. Already in schema, just needs implementation.
`current_verdicts` VIEW not referenced in spec	VIEW on `verdicts` (001)	No	SVC-10 (API/Dashboard) should use `current_verdicts` for email detail views instead of manually `DISTINCT ON`. Already exists in 001.
`rules.rule_group_id` (008) not in spec	`rules` (008)	No	SVC-10 uses `rule_group_id` when displaying rule version history. SVC-04/SVC-08 should include it in rule cache for audit trail. Already in schema.

END OF DOCUMENT — ARCH-SPEC-v2.1 — CyberSiren System Architecture Specification — 2026-03-11

This document is the authoritative reference for implementation. All service boundaries, message schemas, database access patterns, column-level data flows, and migration references defined herein constitute the approved architecture baseline. Items in the Schema Gap section (§15) are tracked implementation deltas.